DAT321/DIT843: Quality Assurance and Testing

Questions about the exam contact:
Examiner: Francisco Gomes, tel. 073 856 4439, francisco.gomes@cse.gu.se
Francisco will visit the exam halls around 09:30 and 11:00.

No aids are allowed for this exam.

The questions in this exam refer to the **ISO 25010:2011** that categorises four types of quality: process, internal, external and quality in use. The exam also covers the eight characteristics for product quality.

Important advice:

- Each question has points assigned shown in the square brackets. When the question is broken down into smaller sub-questions the part of the points for those specific sub-questions are also shown. Example: 1. [10 pts]. 1a. [2 pts] 1b. [8 pts].
- You must write correct, clear, readable, understandable, and unambiguous answers. If we cannot easily read what you are trying to write, then we cannot positively judge your knowledge or your learning of the topic.
- Motivate your answers (a simple statement of facts not answering the question is invalid). Whenever possible refer to modules, features and information about the software product or software development process related to the questions. Avoid circular definitions and vague / generic answers.
- Start each question on a new paper. Sort your answers in order (by question and sub-task) before handing them in. Write only on one side of the paper.
- Write your student code on each page and put the number of the question on every paper. The exam is anonymous, so do not leave any information that can reveal your identity.

The points and the corresponding grade are presented below (100 points in total):

- 00 – 49: U (Fail)
- 50 - 69: 3 (Pass)
- 70 - 84: 4 (Pass with credit)
- 85 - 100: 5 (Pass with distinction)

The exam review is scheduled for:
Date: 2023-01-27 between 13:00 – 14:00.
Location: Jupiter room 400 (Jupiter building 4th floor – Software Engineering)

Question 1. [Total: 30 pts] Using the context below, answer the following questions:

You work for the CarRide-AB company that offers mobility as a service and ride-hailing. **Ride-hailing** is the practice of arranging for travel in a private vehicle driven by its owner for a fee, by means of a website or app (think of apps like **Uber** or Lyft).

There are two types of users: (i) drivers and (ii) passengers. When requesting a ride, passengers indicate their current location, destination and the corresponding fee for the ride; several drivers are then notified about the request in the area and, upon accepting the request, pick up the passenger in their location and drop them off at the chosen destination.

Some features and characteristic of the software product (app) being developed are mentioned below. Whenever we refer to "user" we mean both Drivers and Passenger users:

- **G1:** Users (both passengers and drivers) must register to use the app. Once registered, users must login with username and password.
- **G2:** Drivers must register the information about their vehicle when registering for the app, such as license plate, year, and colour.
- **G3:** The payment of the fee is done by an external online payment service (e.g., BankID linked to a Bank Account).
- **G4:** When paying for a ride, passengers must authorise it via the payment service (BankID).
- **G5:** The passenger can rate the driver by awarding between 1 (very negative experience) and 5 (very positive experience) starts.
- **G6:** Every week, CarRide-AB releases a new version of the app focusing on bug fixing.
- **G7:** Every month, CarRide-AB releases a new version of the app focusing on new features.
- **G8:** The GPS (Global Positioning System) used by your app relies on an external service (e.g., Google Maps) and is not maintained by your team.
- **G9:** The app can be used in different devices such as smartphones, laptops and tablets.
- **G10:** The graphical user interface (GUI) shall be responsive to each device supported.
- **G11:** The payment of the fee must be confirmed by the passenger within 30 seconds upon reaching the destination.

Note: For the questions below, you are allowed to add features or decisions about the software development process (team size, technologies, etc.). However, any assumption related to your answer: (i) must be clearly state, and (ii) cannot contradict any of the points above.

a. **[8 pts]** Choose **two** software product quality attributes, **<u>and, for each,</u>** provide (i) a brief explanation describing the quality attribute, and (ii) why they are relevant to the software product described above. When justifying your choices, you **must** refer to one or more of the points listed above.

There are eight quality attributes, but typically we consider 7, since function suitability is closer to functional requirements of the system.

+4 pts per correct quality attribute choice:

- 2 pts: Correct and clear explanation of attribute + connection to the requirement from example.
- 2 pts: Correct and clear reason to why the attribute is relevant.
- Circular definitions are not acceptable: E.g., "Reliability indicates how reliable the system is".

Some examples are listed below

Reliability: Attribute related to the correct functioning of the system, under specific conditions and periods of time. This is relevant to the system to ensure that drivers and passengers can connect to each other, or that the payment or GPS services are also working whenever a passenger is trying to book a travel. Any of G3, G8, or G11.

Performance efficiency: Attribute related to the resources needed to execute the software product. This is relevant because there are time constraints in the payment. Also, the travel and navigation relies on real-time information. Any of G8 or G11.

Maintainability: Attribute related to the need to change the software product (e.g., add or remove features, fix bugs, etc.). This is relevant due to the need to release new versions of the app. Both G6 and G7 are connected to this quality attribute.

Security: Attribute that ensures the right level of access to the different types of users of the software, and for protecting the information from users and integrity of the app. This is relevant as personal information for identifying users is needed (vehicle data, passenger details, baking and location information, etc.). Some important features are G1 and G2

Usability: Attribute related to how easy it is to find, trigger and operate with the software product. This is relevant because the app needs to be used while driving or users with different needs (e.g., drivers may require accurate navigation while passengers might prefer to easily see information about the driver or travel). Also, the app is used in devices with different user interfaces. An important feature mentioned is: G10 or G9.

Portability: Attribute related to deploying the product in different hardware devices. This is relevant as the software runs in different types of devices, such as smartphones, tablets or laptops. Therefore, this attribute is directly connected to G9.

Compatibility: Attribute related to the ability to share or exchange information between different software applications, products or platforms. This is relevant as other services are needed for the app to function. This attribute is directly connected to the external GPS and Banking services (G3, G4, G8).

b. [7 pts] For each of the quality attributes chosen above create a quality scenario that includes: (i) source, (ii) stimulus, (iii) artefact, (iv) environment, (v) response, and (vi) response measure.

+2 per correct scenario. +3 for the overall quality of both scenarios - i.e., can both scenarios be seen as system level tests.

The scenarios will depend on the chosen attribute. The simplest way to connect to a quality attribute is to use the **response measure** for that. Some examples are:

For performance efficiency:

- Source: GPS Service (or manually by Driver)
- Stimulus: Destination has been reached
- Environment: System is online and operating as expected
- Artefact: Passenger client (or passenger screen)
- Response: Passenger is prompted by a payment dialog screen.
- Response measure: Screen is shown within 10 seconds of the stimulus.

For security:

- Source: A registered driver
- Stimulus: Login attempt
- Environment: Password is incorrect
- Artefact: Driver client (or driver screen)
- Response: Screen shows message of incorrect password. Suggests reset password.
- Response measure: Number of incorrect login attempts.

c. [7 pts] Explain the differences between process quality, internal quality, external quality and quality in use. Focus on the distinguishing aspect between those different types of software qualities.

Your answer cannot be vague or generic. Must focus on the unique aspect about each type of quality. Circular definitions will also not be considered (e.g., "the quality of the product in use", "attributes related to the process", etc.)

+1 pts - Process quality focuses on measures and information related to **development stages or status** of the product such as effort, human resources, number of features completed, etc.
+2 pts - Internal quality focuses on **artefacts or static** information about the **product**
+2 pts - External quality focuses on measures and information related to the **execution or behaviour** of the product.
+2 pts - Quality in use focuses on measures and attributes from the perspective of the **end-user in specific context** of usage.

[8 pts] CarRide-AB wants to promote sustainability in the product that we are creating. Using the Sustainability Awareness Framework, you must (i) **choose or suggest two functionalities for the app** and explain the (ii) immediate, (iii) enabling and (iv) structural effects of those features. You must also indicate the **corresponding dimension of sustainability** in which those effects.

There are many examples. You receive +4 pts per answer. There needs to be a cohesive connection between immediate, enabling and structural effects. We deduct points for effects that are too speculative. Some examples below:

(i) Propose connecting passengers to share rides with the same destination for an adjusted fee.
(ii) The immediate effect is that users will be able to save money in each ride (economical),
(iii) the enabling effect is that we can optimise the vehicle capacity with less rides per vehicle (environment),
(iv) the structural effect is that riders with similar patterns of travels can get to know each other and bond (social).

(i) Propose to adjust routes real-time based on current traffic information and fuel efficiency.
(ii) the immediate effect is the addition of algorithms to figure out new routes (technical),
(iii) the enabling effect is that drivers will find more efficient routes (environment),
(iv) the structural effect is for users to avoid time wasted in traffic jams (individual).

Question 2. [Total: 30 pts] Based on your knowledge of Property-based testing (PBT) and Black-box Testing, answer the questions below. We also listed some examples of methods that can be used when writing your properties.

List of methods that can be used in your properties:

`sum(int[]): int`	Returns the sum of the integers in the specified array.
`length(int[]): int`	Returns the number of elements in the array.
`sort(int[]): int[]`	Returns the array with its values sorted in an ascending order.
`reverse(int[]): int[]`	Returns the array where its values are in reversed positions.
`max(int[]): int`	Returns the highest integer value in the array.
`min(int[]): int`	Returns the lowest integer value in the array.

Specification: The function **`evenOdd(int[])`** **receives** an array of integers and **returns** a new array that contains the exact same numbers as the given array, but rearranged so that all the even numbers come before all the odd numbers. Other than that, the numbers can be in any order. You may modify and return the given array, or make a new array.

Note: Since the implementation is done in Java, there cannot be null elements inside the array of integers.

Below, we list some examples of test cases (inputs and expected outputs) for the SUT above:

T1	`evenOdd([1, 0, 1, 0, 0, 1, 1]) → [0, 0, 0, 1, 1, 1, 1]`
T2	`evenOdd([5, 3, 2]) → [2, 5, 3]`
T3	`evenOdd([2, 2, 2]) → [2, 2, 2]`
T4	`evenOdd([3, 2, 2]) → [2, 2, 3]`
T5	`evenOdd([1, 1, 0, 1, 0]) → [0, 0, 1, 1, 1]`
T6	`evenOdd([1]) → [1]`
T7	`evenOdd([1, 2]) → [2, 1]`
T8	`evenOdd([2, 1]) → [2, 1]`
T9	`evenOdd([]) → []`
T10	`evenOdd(null) → Exception: "Invalid array of integers."`

[10 pts] Create **at least three** properties for the SUT above. You can write the properties using natural language, mathematical notation, or pseudocode. Feel free to use any of the support functions for arrays of integers listed in the beginning of this question.

+3 per property. +1 for the diversity of the properties. Properties should not be redundant (unless they represent very unique cases from a specification perspective).

Note: remember that the properties need to be able to be generated in **sets of input**. It is more likely to use the size of the array, rather than the specific elements in it.

Consider nums[] as the input array.

```
P1: For all length(nums[]) == 1 → evenOdd(nums[]) == nums[]
P2: For all evenOdd(nums[]) = B → evenOdd(B) == B
P3: For all nums[] → evenOdd(nums[]) == evenOdd(reverse(nums[]))

P4: For all nums[] → length(evenOdd(nums[])) == length(nums[])
P5: For all nums[] → sum(evenOdd(nums[])) == sum(nums[])
P6: For all nums[] → min(evenOdd(nums[])) == min(nums[])
P7: For all nums[] → max(evenOdd(nums[])) == max(nums[])
```

- **Note:** Properties P4-P7 are too similar to each other.

```
P8: For all nums[] with all equal elements:evenOdd(nums[]) ==
nums[]
```

```
P9: For all nums[] with all even elements: evenOdd(nums[]) ==
nums[]
P10: For all nums[] with all odd elements: evenOdd(nums[]) ==
nums[]
```

- **Note:** P8-P10 overlap and (can) also overlap with P1.

Note: Properties for the T9 or T10 will not receive full points, as they only generate one input data - not sets.

b. [5 pts] Classify each property above into two types: (1) a property created using one of the patterns for creating properties in PBT (which pattern was used), or (2) a property that is particular for the SUT specification. Justify why does your property falls into that category.

+1 pts per correct classification of property.
+2 for the **overall quality** of justifications

The four patterns are: Symmetry, Invariant, Idempotency and Commutative. The classification depends on your properties. Symmetry and Commutative might be the most rare one (or even not possible), since the array can contain any integer in any order.

From the examples above: P1 is an invariant, P2 is idempotency. All other properties are specification, since the result in the sum, reverse, min, max all depend on the fact that the specification divides the values into even and odd, which is what does not affect the operations.

c. [5 pts] Write one advantage and one limitation of using Property-based Testing.

+ 2 pts for advantage. **+2 pts** for disadvantage. **+1 for clarity** and cohesion of answer. Vague answers should not be allowed (e.g., PBT helps with debugging - ok, how, why?; properties are hard to write - why?)

Some examples of advantages:

- Each property can generate numerous tests. Focus on sets of values, instead of specific ones.
- Helps with debugging because of the minimisation / reduction of failing inputs.
- PBT frameworks offer many operators to instrument the generation of inputs (e.g., data structures with specific constraints on types, values, length, etc.).

Some examples of limitations:

- PBT does not scale as easily for higher levels of testing because of the complexity to represent system-level input or scenarios.
- Since PBT frameworks are typically used at unit-level, their applicability is dependent on the choice of programming language.

d. [10 pts] Explain three differences between Property-based Testing (PBT) and Search-based Software Testing (SBST) techniques.

+3 pts per difference. +1 pt for overall cohesion and clarity of the answers.

Some examples of differences are:

- PBT uses sets of input and a generator, whereas SBST generates specific test cases. Therefore, PBT can create different input values every time that we run the generator, whereas SBST typically generates tests "once" and those will be reused in different testing sessions. (note that this does not mean that SBST will always generate the same tests when running).
- PBT does not assess the quality of an input value, whereas SBST relies on fitness functions to indicate how good a solution (test case or test suite) is.
- SBST relies on metaheuristics and fitness functions to sample (create) test cases, whereas PBT relies on the conditions established by the property.
- Since SBST is guided by metaheuristics, it might get "stuck" in local optimal regions of the fitness, whereas PBT is not guided by a specific function.
- SBST can use fitness functions at the test suite level, hence comparing information from different test cases, whereas PBT focuses on sets of input that are independent from each other (per property).

Question 3. [Total: 40 pts] Using the code below to answer the questions.

The function below parses an array of months of the years (as integers) into the corresponding academic period in a lecture year. There are three academic periods:

(i) The "Spring term" covers the months between January (1) until May (5);
(ii) The "Summer break" happens between June (6) and August (8);
(iii) The "Autumn term" includes the months between September (9) and December (12);

The program can throw two types of Exceptions: the first covers cases where the array of month indices is empty or null, whereas the second cover cases where there is any invalid month index in the input array. Java does not allow null values inside the array.

Note: We added a fault in the code (line 14), which can lead to negative values being classified as "Summer breaks". This was done to trigger the tester in thinking of a partition only for negative values. If you did not find this bug, this should not affect your answers either.

```
1.public String[] parseTermNames(int[] monthIndex) throws Exception{
2.     if(monthIndex == null || monthIndex.length == 0) {
3.      throw new Exception("Error. Month array cannot be empty or null");
4.     }
5.
6.     //Create the code
7.     String[] termName = new String[monthIndex.length];
8.     for(int i = 0; i < monthIndex.length; i++) {.
9.       int currentMonth = monthIndex[i];
10.
11.      if(currentMonth >= 1 && currentMonth < 6){ //January until May
12.             termName[i] = "Spring term";
13.
14.    } else if(currentMonth <= 8){       //June until August
15.             termName[i] = "Summer break";
16.
17.    } else if(currentMonth <= 12){      //September until December
18.             termName[i] = "Autumn term";
19.
20.    } else {
21.       throw new Exception("Invalid month. Month must be within 1:12");
22.      }
23.    }
24.    return termName;
25. }
```

a. [8 pts] A static analysis tools indicates that the `parseTermNames` function above has **Cyclomatic Complexity = 18**, and **Cognitive Complexity = 8** (almost half of the cyclomatic complexity). Using the code above, you must explain (i) what measure is more accurate to represent complexity of the function above, and (ii) why those two measures have so different values.

You might agree or disagree on which measure is more accurate if you explain why you believe the nested control flow (cyclomatic) is indeed harder to understand. We analyse each answer individually.

+3 pts for description of which is accurate
+5 pts for explanation behind different values.

(i) In this case, the cognitive complexity is more accurate, as the algorithm represents a simple interaction of a for-loop over an array to classify its values into three categories. There are no combinations of complex statements.

(ii) Those two measures have different values because the cyclomatic complexity grows significantly for each new if-then block in a nested if-then-else block (e.g., a switch case block) - lines 11 - 20. In contrast, cognitive complexity focuses only on the nesting of different structures and, in this case, there are only two main nested constructs: an if-then-else block within a for-loop structure.

b. **[10 pts]** Create at least four equivalent partition classes for the `parseTermNames` function.

+2 pts per partition. **+2 pts** for **overall diversity** of partitions.

You can use either the values or attributes of the array to define these partitions. The important aspect is that they should not overlap, and it should cover sets of input (it is okay if the set is only one input).

(It is acceptable to have C1 and C2 as the same or different partitions.)

C1: Arrays with at least one value less than or equal to zero:
Outcome: Should throw an exception for the invalid month value.

C2: Arrays with month values greater than 12.
Outcome: Should throw an exception for the invalid month value.

C3: Empty arrays or null arrays.
Outcome: Should throw an exception indicating that the array of integers is not valid.

C4: Arrays with repeated values that are within the valid months.
Outcome: An array with all values in the same term.

C5: Arrays with valid months that map to all possible terms
Outcome: All three different terms are expected to be seen in the output array.

C6: Array with one value between 1 and 5.
Outcome: An array with one element: "Spring term"

C7: Array with one value between 6 and 8.
Outcome: An array with one element: "Summer break"

C8: Array with one value between 9 and 12.
Outcome: An array with one element: "Autumn term"

c. [10 pts] Using the partition classes that you defined above, create test cases that reveal at least three boundaries. You must indicate the corresponding equivalent partition classes of each boundary listed.

Each boundary must have at least two test case, one from each partition. Each test case **must** include the input value and the expected outcome. The input should be "close" enough.

+3 pts per boundary. +1 for overall diversity of boundaries.

Boundary 1: Between Partitions C3 and C2

```
T1: [] → Exception("Error. Month array cannot be null or empty")
T2: [13] → Exception("Invalid month. … within 1:12")
```

Boundary 2: Between Partitions C6 and C7

```
T1: [5] → ["Spring term"]
T2: [6] → ["Summer break"]
```

Boundary 3: Between Partitions C4 and C5

```
T1: [6, 6, 6] → ["Summer break", "Summer break", "Summer break"]
T2: [5, 7, 9] → ["Spring term", "Summer break", "Autumn term"]
```

Boundary 4: Between Partitions C3 and C6

```
T1: [] → Exception("Error. Month array cannot be null or empty")
T2: [10] → ["Autumn term"]
```

d. [7 pts] Explain how test diversity can be used to identify boundaries in a SUT. Illustrate your explanation using at least one of the boundaries chosen above.

+5 pts for clarity of explanation. +2 pts for correctness and clarity of example.

Diversity can be used to find boundaries by comparing the distance values between inputs and the outputs of pairs of tests. Two similar input values (i.e., low distance between them) should reveal similar outputs if they belong to different equivalence partition classes. If we find two similar inputs that lead to two very different outputs, this indicates that the outputs belong to two different partition classes.

Considering the boundaries above, note that Boundary 3 has similar input values (i.e., they only differ in +1-3 values), but the types of terms generated in the output differ from each other (e.g., they differ in the words themselves). An even more explicit case is seen in Boundary 4 where the inputs only "differ by one" (the input has one length +1) but it leads to different **types** of output channels (an exception versus a valid output).

e. [5 pts] For the code above, (i) create a mutant using the Conditional Operation Replacement as a mutation operator. You only need to write the specific line of code that you mutated. (ii) Show an example of a test case that would kill that mutant.

+2 pts for the correct mutant. +3 pts for the correct test case.

Careful **not to mix** with the relational operator where we change symbols like <, >=, ==.

Conditional targets boolean operators (&&, ||, !). The mutant is only applicable in the following lines:

```
Line 2: if(monthIndex == null && monthIndex.length == 0) {
Line 11: if(currentMonth >= 1 || currentMonth < 6){
```

Examples of test cases that would kill both mutants are (must include the expected output):

```
Line 11: if(currentMonth >= 1 || currentMonth < 6){
T1: [12] → ["Autumn term"]
T2: [-5] → Exception (or "Summer break" if u did not "fix" fault)
T3: [7] → ["Summer break"]

Line 2: if(monthIndex == null && monthIndex.length == 0) {
T1: null → Exception("cannot be null or empty")
T2: [] → Exception("cannot be null or empty")
```